**Hochschule Wismar**
University of Applied Sciences
Technology, Business and Design
Fakultät für Ingenieurwissenschaften

# Hackintosh-Datenbank
# Dokumentation

Datenbankgestützte Verwaltung von Hackintosh-Konfigurationen:
Analyse und Implementierung mit MongoDB

Eingereicht am: 05.04.2026

von: Kilian Ketelhohn (520858)
geboren am 27.11.2001 in Schwerin

Betreuer: Prof. Dr. Antje Raab-Düsterhöft

## Aufgabenstellung

# 1 Aufgabenstellung

Das Thema dieser Arbeit beschäftigt sich mit der Installation und dem stabilen Betrieb eines Betriebssystems auf nicht offiziell unterstützter Hardware. Im Fokus steht hierbei insbesondere die Installation von macOS auf Standard-PC-Hardware, die eine Vielzahl hardwareabhängiger Anpassungen erfordert.

Zu diesen Anpassungen zählen unter anderem:

- die Analyse der Hardwarekomponenten und Geräte-IDs,
- die Erstellung individueller ACPI-Anpassungen, wie DSDT- und SSDT-Dateien,
- die Auswahl geeigneter Treiber (Kexts) sowie
- die präzise Konfiguration der Bootloader-Einstellungen.

Ein zentrales Problem besteht darin, dass viele bestehende Anleitungen generische oder vorgefertigte Konfigurationen verwenden. Diese sind oft nicht optimal auf spezifische Systeme abgestimmt, was zu Instabilitäten, fehlender Hardwareunterstützung oder Boot-Problemen führen kann. Daher wird ein systematischer Ansatz erforderlich, bei dem jede Komponente des Systems analysiert und individuell konfiguriert wird.

Ziel der Arbeit ist die Entwicklung einer reproduzierbaren und strukturierten Vorgehensweise, um ein solches System zuverlässig einzurichten. Dabei sollen insbesondere die Aspekte Hardwarekompatibilität, Firmware-Konfiguration, Treiberintegration sowie Bootloader-Einstellungen betrachtet und aufeinander abgestimmt werden.

Als Grundlage dienen unter anderem dokumentierte Leitfäden aus der Hackintosh-Community, wie beispielsweise die Empfehlungen auf `https://dortania.github.io/OpenCore-Install-Guide/prerequisites.html`.

# Inhaltsverzeichnis

**1 Aufgabenstellung** **3**

**1 Einleitung** **7**
1.1 Problemstellung . . . 7
1.2 Zielsetzung . . . 7

**2 Grundlagen** **8**
2.1 Hackintosh . . . 8
2.1.1 Warum Hackintosh? . . . 8
2.1.2 Technische Grundlagen . . . 8
2.2 macOS-Versionen und Hardwarekompatibilität . . . 9
2.2.1 Hardwarekomponenten . . . 9
2.3 ACPI und OpenCore . . . 9
2.3.1 ACPI-Patches . . . 9
2.3.2 OpenCore . . . 9
2.4 MongoDB-Grundlagen . . . 10
2.4.1 Was ist MongoDB? . . . 10
2.4.2 Vorteile für Hackintosh-Projekte . . . 10
2.4.3 Beispielhafte Nutzung . . . 10
2.5 Motivation der Arbeit . . . 10
2.5.1 Ziel dieser Arbeit . . . 10
2.6 Problemstellung und Zielsetzung . . . 11
2.6.1 Problemstellung . . . 11
2.6.2 Zielsetzung . . . 11
2.7 Zusammenfassung für Fachfremde . . . 11

**3 Methodik** **12**
3.1 Vorgehensweise . . . 12
3.2 Konzept des Datenbankmodells . . . 12
3.2.1 Device-Modell . . . 12
3.2.2 Designentscheidungen . . . 13
3.3 Zusammenfassung . . . 13

**4 Praktische Umsetzung: Web-Frontend** **14**
4.1 Einleitung . . . 14
4.2 Architekturübersicht . . . 14
4.3 Technologien im Detail . . . 14
4.3.1 MongoDB . . . 14
4.3.2 React . . . 15
4.3.3 Web-Interface . . . 15

4.4 Implementierungsschritte . . . 16
4.4.1 Backend-Anbindung . . . 16
4.4.2 Frontend-Entwicklung . . . 16
4.4.3 Testing und Validierung . . . 16
4.5 Zusammenfassung . . . 16

**5 Praktische Umsetzung: Backend 18**
5.1 Einleitung . . . 18
5.2 Architektur des Backends . . . 18
5.3 Technologien im Detail . . . 18
5.3.1 Node.js . . . 18
5.3.2 Express . . . 19
5.3.3 MongoDB . . . 19
5.4 API-Design . . . 19
5.4.1 Beispiel-Endpunkte . . . 19
5.5 Datenfluss und Verarbeitung . . . 20
5.6 Sicherheitsaspekte . . . 20
5.7 Testing und Wartung . . . 20
5.8 Zusammenfassung . . . 20

**6 Praktische Umsetzung: Datenbank 22**
6.1 Einleitung . . . 22
6.2 Datenmodell und Collections . . . 22
6.2.1 Struktur eines Config-Dokuments . . . 22
6.2.2 Erklärung der Felder . . . 23
6.3 Abfragen und Filter . . . 23
6.4 Indexierung und Performance . . . 24
6.4.1 Beispiel: Index erstellen . . . 24
6.5 Praktischer Nutzen . . . 24
6.6 Zusammenfassung . . . 24

**7 Praktische Umsetzung: Implementierung 25**
7.1 Einleitung . . . 25
7.2 Workflow der Implementierung . . . 25
7.3 Integration der Komponenten . . . 25
7.3.1 Frontend-Backend-Kommunikation . . . 25
7.3.2 Backend-Datenbank-Schnittstelle . . . 26
7.3.3 Gesamtsystem . . . 26
7.4 Praktische Umsetzungsschritte . . . 26
7.5 Testing und Qualitätssicherung . . . 27
7.6 Besondere Herausforderungen . . . 27
7.7 Zusammenfassung . . . 27
7.8 Einsatz von KI zur Generierung von Dummy-Daten . . . 28
7.8.1 Motivation . . . 28
7.8.2 KI-gestützte Datengenerierung . . . 28
7.8.3 Integration in MongoDB . . . 29
7.8.4 Automatisierte Generierung mehrerer Datensätze . . . 30

7.8.5 Validierung der generierten Daten . . . . . 30
7.8.6 Vorteile des KI-Einsatzes . . . . . 30
7.8.7 Grenzen und Herausforderungen . . . . . 31
7.8.8 Fazit . . . . . 31

**8 Evaluation** **32**
8.1 Testdaten . . . . . 32
8.2 Performance . . . . . 32
8.3 Ergebnisse . . . . . 32

**9 Fazit** **33**
9.1 Zusammenfassung . . . . . 33
9.2 Ausblick . . . . . 33

**Abbildungsverzeichnis** **37**

**Tabellenverzeichnis** **38**

**Listings** **39**

**Abkürzungsverzeichnis** **40**

**Selbstständigkeitserklärung** **41**

# 1 Einleitung

Die Installation von macOS auf nicht offiziell unterstützter Hardware, sogenannte *Hackintosh*-Systeme, gewinnt zunehmend an Bedeutung in der Forschung und bei Enthusiasten. Herausforderungen ergeben sich insbesondere durch inkompatible Hardwarekomponenten, unterschiedliche macOS-Versionen und die komplexe Konfiguration von Bootloadern wie OpenCore.

Ziel dieser Arbeit ist es, eine strukturierte Datenbank zur Verwaltung von Hackintosh-Konfigurationen zu entwickeln, die eine einheitliche Erfassung, Abfrage und Analyse ermöglicht. Die Motivation liegt darin, den Installationsaufwand zu reduzieren, die Fehleranfälligkeit zu minimieren und eine zentrale Wissensbasis für unterschiedliche Hardwarekombinationen bereitzustellen.

## 1.1 Problemstellung

Viele Anwender verlieren beim Hackintosh-Prozess Zeit durch Trial-and-Error. Fehlende Dokumentation über kompatible Komponenten erschwert die Reproduzierbarkeit von Setups. Eine zentrale, strukturierte Datenbank soll diese Lücke schließen.

## 1.2 Zielsetzung

- Entwicklung eines MongoDB-Datenmodells für Hackintosh-Geräte.
- Implementierung eines Web-Frontends für einfache Eingabe und Abfrage.
- Bereitstellung von Dummy-Daten zur Evaluation der Datenbankperformance.

# 2 Grundlagen

## 2.1 Hackintosh

Ein *Hackintosh* bezeichnet einen PC, auf dem macOS auf Hardware ausgeführt wird, die von Apple offiziell nicht unterstützt wird. Im Gegensatz zu einem echten Mac, der Hardware und Software optimal aufeinander abstimmt, muss bei einem Hackintosh der Anwender viele Anpassungen selbst vornehmen.

### 2.1.1 Warum Hackintosh?

Die Motivation hinter einem Hackintosh kann vielfältig sein:

- Nutzung von macOS auf eigener, oft leistungsfähiger Hardware
- Kosteneinsparung gegenüber einem Apple Mac
- Experimentelle Forschungsprojekte oder Softwareentwicklung, die macOS erfordern

### 2.1.2 Technische Grundlagen

Damit macOS auf einem normalen PC läuft, sind Anpassungen notwendig:

- **Bootloader:** Ein Programm, das den Start von macOS auf dem PC ermöglicht. OpenCore ist der aktuelle Standard. Es ttäuschtmmacOS vor, dass die Hardware offiziell unterstützt wird.
- **ACPI-Patches:** ACPI (Advanced Configuration and Power Interface) regelt die Kommunikation zwischen Hardware und Betriebssystem. Patches korrigieren fehlerhafte oder fehlende Hardwareinformationen.
- **Kexts (Kernel Extensions):** Treiber für macOS, die bestimmte Hardware erst funktionsfähig machen.

## 2.2 macOS-Versionen und Hardwarekompatibilität

macOS entwickelt sich ständig weiter. Jede Version bringt neue Features, verändert Treiberanforderungen und unterstützt neue Hardware. Die wichtigsten Versionen für Hackintosh sind:

- **Monterey, Ventura, Sonoma** – jeweils mit eigenen Anforderungen

### 2.2.1 Hardwarekomponenten

Nicht jede CPU, GPU oder WLAN-Karte funktioniert unter macOS. Deshalb ist die Wahl der Komponenten entscheidend:

- **CPU:** Intel- und AMD-Prozessoren sind möglich, Apple Silicon (M1, M2) nur auf Macs.
- **GPU:** macOS unterstützt offiziell Intel-, NVIDIA- und AMD-Karten. Treiber für NVIDIA sind oft problematisch.
- **WLAN-Karten:** Einige Karten funktionieren nativ (z.B. Broadcom), andere benötigen Kexts.

Für den Laien: Man kann sich vorstellen, dass macOS nur "bestimmte Hardware spricht"– alles andere muss man quasi übersetzen.

## 2.3 ACPI und OpenCore

### 2.3.1 ACPI-Patches

ACPI regelt z.B. Stromversorgung, Lüftersteuerung und Sleep-Modus. Patches sind kleine Anpassungen, die macOS sagen, wie es die Hardware korrekt nutzen soll.

### 2.3.2 OpenCore

OpenCore ist ein moderner Bootloader:

- Startet macOS auf nicht unterstützter Hardware
- Emuliert eine von Apple unterstützte Hardwareumgebung
- Ermöglicht individuelle Konfigurationen für verschiedene Hardwarekomponenten

## 2.4 MongoDB-Grundlagen

### 2.4.1 Was ist MongoDB?

**MongoDB** ist eine *NoSQL-Datenbank*. Im Gegensatz zu klassischen SQL-Datenbanken, die Tabellen mit festen Spalten nutzen, speichert MongoDB Daten als flexible Dokumente, ähnlich wie JSON-Dateien.

### 2.4.2 Vorteile für Hackintosh-Projekte

- Flexibles Schema: verschiedene Hardwaretypen und Versionen lassen sich einfach speichern
- Dokumentenorientiert: jede Konfiguration wird als ein einzelnes Dokument gespeichert
- Abfragen und Indizes: erleichtern die Suche nach kompatibler Hardware

### 2.4.3 Beispielhafte Nutzung

Man kann z.B. alle PCs mit Intel-CPU, AMD-GPU und macOS Ventura filtern. MongoDB erlaubt schnelle, flexible Abfragen ohne komplizierte Tabellenverknüpfungen.

## 2.5 Motivation der Arbeit

Die Installation von macOS auf nicht unterstützter Hardware wird für Forschung, Entwicklung und Enthusiasten zunehmend relevant. Probleme treten auf, weil:

- Hardware nicht offiziell unterstützt wird
- macOS-Versionen unterschiedliche Anforderungen haben
- Bootloader und Kexts komplex konfiguriert werden müssen

### 2.5.1 Ziel dieser Arbeit

Entwicklung einer zentralen Datenbank, die:

- Hackintosh-Konfigurationen einheitlich erfasst
- Abfragen und Analysen ermöglicht
- Installation vereinfacht und Fehler reduziert

## 2.6 Problemstellung und Zielsetzung

### 2.6.1 Problemstellung

Viele Anwender verbringen beim Hackintosh-Prozess unnötig viel Zeit mit Trial-and-Error. Fehlende Dokumentation über kompatible Hardware erschwert die Reproduzierbarkeit.

### 2.6.2 Zielsetzung

- Entwicklung eines MongoDB-Datenmodells für Hackintosh-Geräte
- Erstellung eines Web-Frontends für einfache Eingabe und Abfrage
- Bereitstellung von Dummy-Daten zur Evaluierung der Datenbankperformance

## 2.7 Zusammenfassung für Fachfremde

Hackintosh-Systeme erlauben macOS auf PCs, sind aber kompliziert. ACPI-Patches, Kexts und OpenCore sorgen dafür, dass das System überhaupt läuft. Eine Datenbank wie MongoDB hilft, diese Konfigurationen zu dokumentieren, durchsuchen und wiederverwenden – wie ein digitales Logbuch für verschiedene Hardware-Setups.

# 3 Methodik

## 3.1 Vorgehensweise

Die Vorgehensweise dieser Arbeit folgt einem systematischen Ansatz, um die Komplexität von Hackintosh-Konfigurationen zu strukturieren und eine nutzbare Datenbasis zu schaffen. Die Arbeit gliedert sich in drei Hauptschritte:

1. **Analyse der Hardwarelandschaft:** Es wurde eine umfassende Bestandsaufnahme gängiger Hackintosh-kompatibler Hardware durchgeführt, inklusive Prozessoren, Grafikkarten, WLAN-Modulen und Bootloader-Versionen. Ziel war es, typische Konfigurationsmuster und problematische Kombinationen zu identifizieren.
2. **Definition eines Datenmodells:** Auf Grundlage der Analyse wurde ein flexibles, dokumentenorientiertes Datenmodell erstellt, das die unterschiedlichen Geräte- und Konfigurationsparameter abbildet. Besonderes Augenmerk lag auf der Erweiterbarkeit für neue Hardware und Versionen von macOS.
3. **Praktische Umsetzung der Datenerfassung:** Aufbau einer Webanwendung zur Eingabe, Speicherung und Abfrage von Konfigurationen. Die Methodik stellt sicher, dass Daten konsistent, reproduzierbar und nutzerfreundlich erfasst werden können.

## 3.2 Konzept des Datenbankmodells

Das Datenbankmodell ist auf die Speicherung heterogener Hackintosh-Konfigurationen optimiert. Jedes Gerät bzw. jede Konfiguration wird als Dokument abgebildet, das alle relevanten Eigenschaften enthält.

### 3.2.1 Device-Modell

Die zentrale Einheit des Modells ist das Device-Dokument, das folgende Felder umfasst:

| Feld | Beschreibung |
|---|---|
| device_name | Name oder Bezeichnung des Geräts |
| manufacturer | Hersteller |
| cpu | Prozessor-Modell |
| gpu | Grafikkarte / Grafikchip |
| wifi | WLAN-Modul oder -Karte |
| macos_version | Unterstützte macOS-Version(en) |
| status | Stabilität der Konfiguration (z.B. stabil, instabil) |
| opencore_version | Verwendete OpenCore-Version |
| kexts | Liste der eingesetzten Kernel Extensions |
| acpi_patches | Liste der angewendeten ACPI-Patches |
| config_plist | Referenz oder Pfad zur Konfigurationsdatei |
| bios_settings | BIOS-/UEFI-Einstellungen für Kompatibilität |
| known_issues | Bekannte Probleme oder Einschränkungen |

**Tabelle 1:** Felder des Device-Modells für Hackintosh-Konfigurationen

### 3.2.2 Designentscheidungen

- **Flexibilität:** Durch die dokumentenorientierte Struktur können neue Hardwaretypen, Patches oder macOS-Versionen ohne aufwändige Migrationen hinzugefügt werden.
- **Reproduzierbarkeit:** Jede Konfiguration ist vollständig dokumentiert, wodurch sie von anderen Nutzern nachvollzogen werden kann.
- **Erweiterbarkeit:** Das Modell erlaubt zukünftige Ergänzungen, z.B. Benutzerrollen, Bewertungen oder detaillierte Leistungsmetriken.

## 3.3 Zusammenfassung

Die Methodik stellt sicher, dass sowohl die Analyse der Hardware als auch die Umsetzung in ein flexibles, erweiterbares Datenmodell wissenschaftlich fundiert erfolgt. Durch die klar definierte Vorgehensweise und die strukturierte Dokumentation der Device-Parameter wird die Grundlage für eine zentrale, reproduzierbare Wissensbasis geschaffen, die die spätere Implementierung in der Webanwendung unterstützt. graphicx

# 4 Praktische Umsetzung: Web-Frontend

## 4.1 Einleitung

Für die Umsetzung der Hackintosh-Datenbank wurde ein Web-Frontend entwickelt, das die Eingabe, Verwaltung und Abfrage von Konfigurationen ermöglicht. Die Wahl eines Web-Frontends erlaubt es, das System plattformunabhängig über jeden modernen Browser zu nutzen. Das Projekt kombiniert eine **MongoDB**-Datenbank für die Speicherung der Konfigurationen mit einem **React**-basierten Frontend, das eine intuitive Benutzeroberfläche bereitstellt.

## 4.2 Architekturübersicht

Die Architektur des Systems folgt einem klassischen *Client-Server*-Modell:

- **Frontend (Client):** Entwickelt mit React, zeigt Daten an und ermöglicht Interaktion
- **Backend (Server):** Vermittelt zwischen Frontend und MongoDB, verarbeitet Anfragen und liefert Ergebnisse zurück
- **Datenbank:** MongoDB speichert die Hackintosh-Konfigurationen in dokumentenorientierter Form

## 4.3 Technologien im Detail

### 4.3.1 MongoDB

MongoDB dient als *NoSQL*-Datenbank, in der jede Hackintosh-Konfiguration als einzelnes Dokument gespeichert wird. Vorteile:

- Flexible Speicherung von verschiedenen Hardwarekombinationen
- Einfaches Erweitern der Datenstruktur ohne komplizierte Migrationen
- Schnelle Abfragen durch Indizes und Aggregationsfunktionen

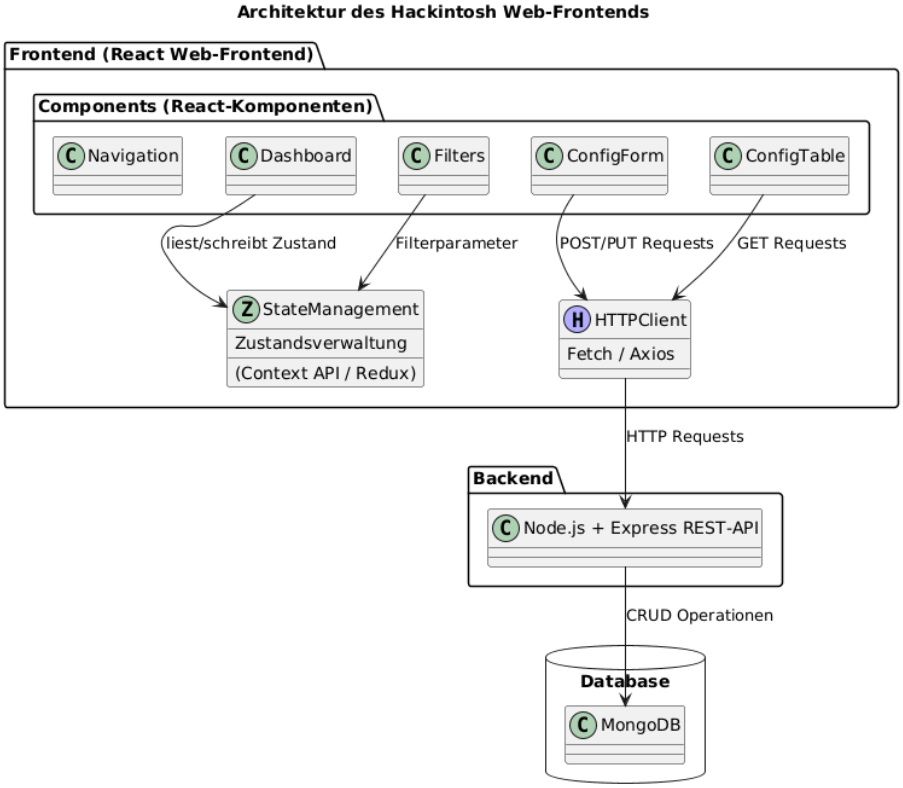


**Bild 1:** Übersicht der Systemarchitektur

### 4.3.2 React

**React** ist ein JavaScript-Framework für die Erstellung dynamischer Benutzeroberflächen. Hauptvorteile:

- Komponentenbasierte Entwicklung: jede Funktion oder Anzeige wird modular erstellt
- Reaktive Darstellung: Änderungen in der Datenbank werden sofort im Frontend sichtbar
- Einfaches Einbinden von Formularen, Tabellen und Filterfunktionen

### 4.3.3 Web-Interface

Das Frontend bietet eine intuitive Oberfläche:

- **Eingabeformulare:** Neue Hackintosh-Konfigurationen können einfach hinzugefügt werden
- **Filter- und Suchfunktionen:** Benutzer können z.B. nach CPU, GPU oder macOS-Version filtern

- **Visualisierung:** Ergebnisse werden übersichtlich in Tabellen oder Kartenansichten dargestellt

Das Interface ist responsiv, also für Desktop, Tablet und Smartphone optimiert.

## 4.4 Implementierungsschritte

### 4.4.1 Backend-Anbindung

1. Einrichtung der MongoDB-Datenbank und Erstellung der notwendigen Collections
2. Implementierung eines REST-APIs mit Node.js und Express zur Kommunikation mit dem Frontend
3. Definition von Endpunkten für CRUD-Operationen (Create, Read, Update, Delete)

### 4.4.2 Frontend-Entwicklung

1. Aufbau der Projektstruktur mit React-Komponenten
2. Erstellung von Formularen zur Eingabe von Konfigurationsdaten
3. Implementierung von Tabellen- und Filterfunktionen zur Anzeige der Daten
4. Anbindung an das REST-API zur dynamischen Datenaktualisierung

### 4.4.3 Testing und Validierung

- Testen der Datenbankanfragen mit Dummy-Daten
- Sicherstellen, dass alle Filter- und Suchfunktionen korrekt arbeiten
- Überprüfung der Benutzerfreundlichkeit auf verschiedenen Geräten

## 4.5 Zusammenfassung

Durch die Kombination von MongoDB und React konnte ein flexibles, benutzerfreundliches Web-Frontend erstellt werden, das:

- Eine zentrale Verwaltung von Hackintosh-Konfigurationen ermöglicht
- Eingabe, Suche und Analyse intuitiv und schnell macht
- Auf verschiedenen Endgeräten genutzt werden kann

Das Frontend bildet damit das praktische Kernstück des Projekts und zeigt, wie theoretische Datenbankstrukturen im Alltag angewendet werden können. graphicx

# 5 Praktische Umsetzung: Backend

## 5.1 Einleitung

Das Backend bildet das Herzstück des Projekts und übernimmt die gesamte Kommunikation zwischen dem Web-Frontend und der **MongoDB**-Datenbank. Es sorgt dafür, dass Daten zuverlässig gespeichert, abgerufen, aktualisiert und gelöscht werden können. Für dieses Projekt wurde das Backend mit **Node.js** und **Express** umgesetzt, da diese Technologien leichtgewichtig, modular und weit verbreitet sind und sich besonders für skalierbare Webanwendungen eignen.

## 5.2 Architektur des Backends

Die Architektur des Backends folgt einem klassischen *Client-Server-Modell*:

- **Server:** Empfängt und verarbeitet HTTP-Anfragen vom Frontend.
- **API-Endpunkte:** Definierte URLs, über die Daten abgefragt oder verändert werden können.
- **Datenbank-Anbindung:** MongoDB speichert sämtliche Hackintosh-Konfigurationen in dokumentenorientierter Form.

## 5.3 Technologien im Detail

### 5.3.1 Node.js

Node.js ist eine serverseitige JavaScript-Laufzeitumgebung:

- Ermöglicht die Ausführung von JavaScript außerhalb des Browsers.
- Unterstützt die Erstellung skalierbarer, leistungsfähiger Webanwendungen.
- Ideal für asynchrone Verarbeitung von Anfragen, wie sie bei Web-APIs üblich ist.

### 5.3.2 Express

Express ist ein minimalistisches Webframework für Node.js:

- Vereinfacht die Erstellung von Routen und API-Endpunkten.
- Bietet Middleware-Funktionalitäten für Logging, Fehlerbehandlung und Authentifizierung.
- Ermöglicht modulare Strukturierung des Backends.

### 5.3.3 MongoDB

MongoDB ist eine dokumentenorientierte NoSQL-Datenbank:

- Speichert Hackintosh-Konfigurationen in flexiblen Dokumenten anstelle starrer Tabellen.
- Unterstützt dynamische Datenstrukturen, wodurch verschiedene Hardwarekombinationen einfach abgebildet werden können.
- Bietet leistungsstarke Abfrage- und Aggregationsmöglichkeiten.

## 5.4 API-Design

Das Backend stellt ein REST-API bereit. Jeder Endpunkt erfüllt grundlegende CRUD-Funktionen:

- **Create:** Neue Konfigurationen anlegen.
- **Read:** Bestehende Konfigurationen abrufen, optional nach Filterkriterien.
- **Update:** Bestehende Konfigurationen bearbeiten.
- **Delete:** Konfigurationen löschen.

### 5.4.1 Beispiel-Endpunkte

- `GET /configs` – Alle Konfigurationen abrufen.
- `GET /configs/:id` – Einzelne Konfiguration abrufen.
- `POST /configs` – Neue Konfiguration anlegen.
- `PUT /configs/:id` – Bestehende Konfiguration aktualisieren.
- `DELETE /configs/:id` – Konfiguration löschen.

## 5.5 Datenfluss und Verarbeitung

1. Das Frontend sendet eine Anfrage an einen API-Endpunkt.
2. Das Backend prüft die Eingaben und bereitet sie für die Datenbank auf.
3. MongoDB speichert die Daten oder liefert die angeforderten Ergebnisse zurück.
4. Das Backend formatiert die Antwort als JSON und sendet sie an das Frontend zurück.

## 5.6 Sicherheitsaspekte

Auch in einem prototypischen Projekt wurden grundlegende Sicherheitsmaßnahmen umgesetzt:

- Validierung von Benutzereingaben, um fehlerhafte oder schädliche Daten zu verhindern.
- Strukturierte Fehlerbehandlung, um Abstürze oder unerwartete Antworten zu vermeiden.
- Einsatz von Middleware für Logging und optional Authentifizierung.

## 5.7 Testing und Wartung

- Testen aller API-Endpunkte mit Dummy-Daten.
- Sicherstellen, dass fehlerhafte Eingaben korrekt verarbeitet werden.
- Beobachtung der Datenbankperformance bei wachsender Datenmenge.
- Modularer Aufbau erleichtert zukünftige Erweiterungen oder Änderungen.

## 5.8 Zusammenfassung

Das Backend bildet die zentrale Schnittstelle zwischen Frontend und Datenbank. Mit Node.js, Express und MongoDB konnte ein leichtgewichtiges, skalierbares System erstellt werden, das eine zuverlässige Verwaltung von Hackintosh-Konfigurationen ermöglicht. Die klare Trennung zwischen API-Endpunkten, Datenbankzugriff und Serverlogik bildet eine solide Grundlage für Weiterentwicklungen und Anpassungen.

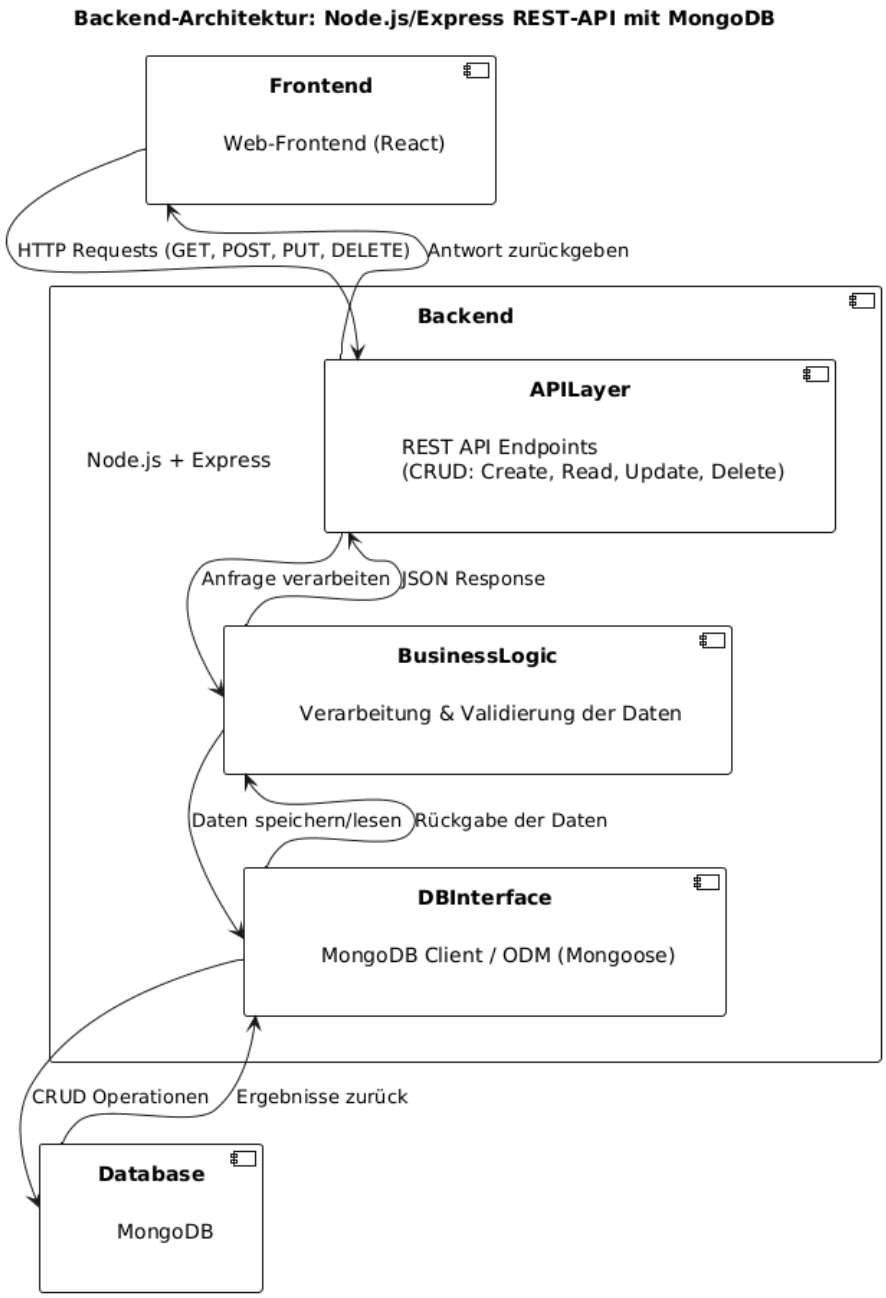


**Bild 2:** Übersicht des Backends und Datenflusses zwischen Frontend und Datenbank

# 6 Praktische Umsetzung: Datenbank

## 6.1 Einleitung

Die zentrale Datenhaltung des Projekts erfolgt über **MongoDB**, eine dokumentenorientierte NoSQL-Datenbank. MongoDB eignet sich besonders für Hackintosh-Konfigurationen, da diese heterogene Daten enthalten: unterschiedliche CPU-, GPU- und WLAN-Karten-Kombinationen, macOS-Versionen und individuelle Anpassungen. Die flexible Datenstruktur ermöglicht es, neue Hardware oder Parameter ohne aufwändige Migrationen zu integrieren.

## 6.2 Datenmodell und Collections

In MongoDB werden Daten nicht in starren Tabellen gespeichert, sondern als *Dokumente* in *Collections*. Jede Collection entspricht in etwa einer Tabelle in relationalen Datenbanken, und jedes Dokument einem Datensatz.

Für das Projekt wurden folgende Collections definiert:

- **configs:** Speichert die Hackintosh-Konfigurationen.
- **users (optional):** Speichert Benutzerdaten, z.B. für ein zukünftiges Authentifizierungssystem.

### 6.2.1 Struktur eines Config-Dokuments

Ein Beispiel für ein Dokument in der `configs`-Collection:

```
{
    "_id": ObjectId("..."),
    "name": "Mein Hackintosh",
    "cpu": "Intel Core i7-10700K",
    "gpu": "AMD Radeon RX 580",
    "wlan": "Broadcom BCM94360",
```

```
    "macos_version": "Ventura",
    "bootloader": "OpenCore",
    "acpi_patches": [
        "DSDT-Patch1",
        "SSDT-Patch2"
    ],
    "kexts": [
        "Lilu.kext",
        "WhateverGreen.kext"
    ],
    "notes": "System stabil, Sleep funktioniert"
}
```

### 6.2.2 Erklärung der Felder

- **_id:** Automatisch generierte eindeutige ID für jedes Dokument.
- **name:** Individueller Name der Hackintosh-Konfiguration.
- **cpu, gpu, wlan:** Hardwarekomponenten.
- **macos_version:** Installierte macOS-Version.
- **bootloader:** Genutzter Bootloader (z.B. OpenCore).
- **acpi_patches, kexts:** Listen der eingesetzten Patches und Treiber.
- **notes:** Freitextfeld für zusätzliche Informationen.

## 6.3 Abfragen und Filter

MongoDB ermöglicht flexible Abfragen über JSON-ähnliche Syntax:

- Alle Konfigurationen mit Intel-CPU:

  ```
  db.configs.find({ "cpu": /Intel/ })
  ```

- Konfigurationen mit macOS Ventura und AMD-GPU:

  ```
  db.configs.find({ "macos_version": "Ventura", "gpu": /AMD/ })
  ```

- Nur bestimmte Felder zurückgeben:

```
db.configs.find({ "cpu": /Intel/ }, { "name": 1, "gpu": 1 })
```

## 6.4 Indexierung und Performance

Für schnelle Abfragen wurden Indexe auf häufig gefilterte Felder gesetzt:

- `cpu`
- `gpu`
- `macos_version`

### 6.4.1 Beispiel: Index erstellen

```
db.configs.createIndex({ "cpu": 1, "gpu": 1, "macos_version": 1 })
```

## 6.5 Praktischer Nutzen

Die Datenbank ermöglicht:

- Zentrale, strukturierte Verwaltung von Hackintosh-Konfigurationen
- Flexible Abfrage und Filterung nach Hardware und macOS-Version
- Leichte Erweiterung bei neuen Hardwarekomponenten
- Grundlage für das React-Frontend, um Daten dynamisch anzuzeigen

## 6.6 Zusammenfassung

MongoDB bildet das Rückgrat des Projekts. Die dokumentenorientierte Speicherung erlaubt eine flexible, skalierbare und leicht erweiterbare Verwaltung von Hackintosh-Systemen. Durch die Kombination von klar definierten Collections, sinnvoll strukturierten Dokumenten und leistungsfähigen Abfragen entsteht eine zentrale Wissensbasis für Hardware-Konfigurationen, die sowohl von Menschen als auch vom Web-Frontend effizient genutzt werden kann. graphicx

# 7 Praktische Umsetzung: Implementierung

## 7.1 Einleitung

In diesem Kapitel wird beschrieben, wie die einzelnen Komponenten des Projekts zu einem funktionierenden Gesamtsystem integriert wurden. Im Vordergrund steht die Beschreibung des **Workflows**, der **Integration von Frontend, Backend und Datenbank** sowie der **praktischen Umsetzungsschritte**, ohne die bereits detailliert beschriebenen Technologien zu wiederholen.

## 7.2 Workflow der Implementierung

Die Implementierung erfolgte in mehreren aufeinanderfolgenden Schritten:

1. **Definition des Datenmodells:** Die Struktur der Hackintosh-Konfigurationen wurde in MongoDB festgelegt.
2. **Aufbau des Backends:** REST-API-Endpunkte wurden implementiert, um CRUD-Operationen auf den Daten zu ermöglichen.
3. **Entwicklung des Frontends:** React-Komponenten wurden erstellt, um Daten anzuzeigen, einzugeben und zu filtern.
4. **Integration:** Frontend und Backend wurden über HTTP-Anfragen verbunden; Datenbankabfragen wurden über das Backend gesteuert.
5. **Testen:** Alle Komponenten wurden schrittweise mit Dummy-Daten überprüft, um Funktionsfähigkeit, Datenkonsistenz und Performance sicherzustellen.

## 7.3 Integration der Komponenten

### 7.3.1 Frontend-Backend-Kommunikation

Die React-Komponenten kommunizieren über HTTP-Requests mit den REST-API-Endpunkten des Node.js-Backends. Dabei werden Daten als JSON-Objekte übertragen. Änderungen im Frontend führen zu entsprechenden Anfragen an die API, die die Datenbank aktualisiert.

### 7.3.2 Backend-Datenbank-Schnittstelle

Das Backend stellt die einzige Schnittstelle zur MongoDB-Datenbank dar. Dies garantiert:

- Einheitliche Datenvalidierung
- Kontrollierte Schreib- und Lesezugriffe
- Konsistente Datenhaltung zwischen mehreren Benutzern

### 7.3.3 Gesamtsystem

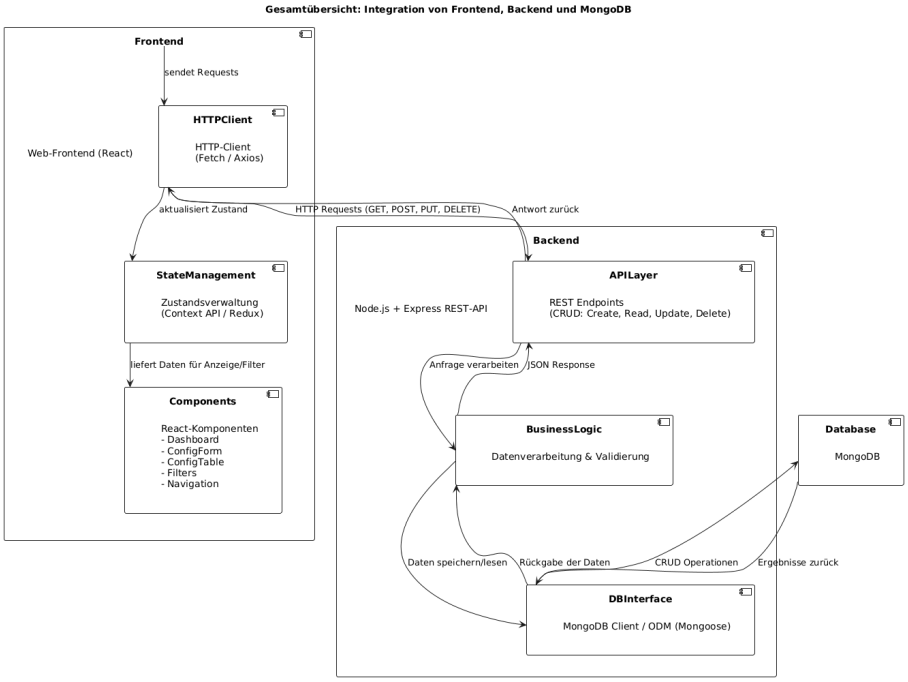


**Bild 3:** Gesamtübersicht des implementierten Systems

## 7.4 Praktische Umsetzungsschritte

Die konkrete Umsetzung umfasste:

- Erstellung der MongoDB-Collections und Indizes
- Implementierung von Backend-Routen für CRUD-Operationen
- Aufbau der React-Komponenten inkl. Formularen, Tabellen und Filterfunktionen
- Verbindung von Frontend und Backend über Fetch- bzw. Axios-Requests

- Implementierung einfacher Validierungen auf Frontend- und Backend-Seite
- Nutzung von Dummy-Daten zur Überprüfung der Funktionsfähigkeit und Performance

## 7.5 Testing und Qualitätssicherung

Die Qualitätssicherung erfolgte iterativ:

- Unit-Tests für einzelne Backend-Endpunkte
- Funktionale Tests der Frontend-Komponenten mit realistischen Dummy-Daten
- Überprüfung der Datenkonsistenz zwischen Frontend, Backend und Datenbank
- Performance-Tests bei steigender Datenmenge

## 7.6 Besondere Herausforderungen

Während der Implementierung traten insbesondere folgende Herausforderungen auf:

- Sicherstellung der Synchronisation zwischen Frontend-Darstellung und Backend-Daten
- Handhabung heterogener Hardwarekonfigurationen in der Datenbank
- Absicherung gegen fehlerhafte Eingaben und inkonsistente Daten
- Modularisierung des Codes, um spätere Erweiterungen zu erleichtern

## 7.7 Zusammenfassung

Die Implementierung vereint Frontend, Backend und MongoDB zu einem funktionierenden System. Durch die klare Trennung von Komponenten, standardisierte API-Endpunkte und ein flexibles Datenmodell konnte eine stabile, erweiterbare und nutzerfreundliche Plattform geschaffen werden. Die iterative Vorgehensweise und umfassende Tests stellten sicher, dass das System zuverlässig funktioniert und als Grundlage für zukünftige Erweiterungen genutzt werden kann.

## 7.8 Einsatz von KI zur Generierung von Dummy-Daten

Im Rahmen der Entwicklung und Evaluation der Hackintosh-Datenbank ist es notwendig, eine ausreichende Menge an Testdaten bereitzustellen. Da reale Konfigurationsdaten nicht immer in ausreichendem Umfang verfügbar sind, können KI-gestützte Verfahren genutzt werden, um realistische Dummy-Daten zu generieren und automatisiert in die MongoDB-Datenbank einzupflegen.

### 7.8.1 Motivation

Die Generierung von Dummy-Daten erfüllt mehrere Zwecke:

- Testen der Datenbankstruktur und -konsistenz
- Evaluation von Such- und Filterfunktionen
- Simulation realer Hardwarekonfigurationen
- Performance-Analyse von Abfragen

Durch den Einsatz von KI lassen sich strukturierte und gleichzeitig variantenreiche Datensätze erzeugen, die reale Hackintosh-Systeme nachbilden.

### 7.8.2 KI-gestützte Datengenerierung

Zur Generierung der Dummy-Daten kann ein KI-Modell verwendet werden, das auf Basis definierter Eingabestrukturen vollständige Datensätze erzeugt. Die Eingabe erfolgt typischerweise als Prompt, der das gewünschte Schema beschreibt.

Ein Beispiel für einen generierten Datensatz ist in Listing 1 dargestellt.

```
{
  "cpu": "Intel Core i7-9700K",
  "gpu": "AMD Radeon RX 580",
  "mainboard": "ASUS Prime Z390-A",
  "ram": "32GB DDR4",
  "storage": "1TB NVMe SSD",
  "macOS_version": "Ventura",
  "bootloader": "OpenCore",
  "status": "stable",
  "notes": "Erfolgreiche Installation mit minimalen Anpassungen",
  "acpi_patches": ["SSDT-EC", "SSDT-PLUG"],
  "kexts": ["Lilu", "WhateverGreen", "VirtualSMC"]
}
```

**Listing 1:** Beispiel eines KI-generierten Dummy-Datensatzes

Die KI erzeugt dabei konsistente Kombinationen aus Hardwarekomponenten und passenden Konfigurationen, was insbesondere für Hackintosh-Systeme von Bedeutung ist.

### 7.8.3 Integration in MongoDB

Die generierten Daten können automatisiert in die MongoDB-Datenbank eingefügt werden. Dies erfolgt typischerweise über ein Node.js-Backend.

Ein Beispiel für das Einfügen eines Datensatzes ist in Listing 2 dargestellt.

```
const { MongoClient } = require("mongodb");

async function insertDummy() {
    const client = new MongoClient("mongodb://localhost:27017");

    try {
        await client.connect();
        const db = client.db("hackintoshDB");
        const collection = db.collection("configs");

        const dummyData = {
            cpu: "Intel Core i7-9700K",
            gpu: "AMD Radeon RX 580",
            mainboard: "ASUS Prime Z390-A",
            ram: "32GB DDR4",
            macOS_version: "Ventura",
            status: "stable"
        };

        await collection.insertOne(dummyData);
        console.log("Dummy-Daten erfolgreich eingef gt");
    } finally {
        await client.close();
    }
}

insertDummy();
```

**Listing 2:** Einfügen eines KI-generierten Datensatzes in MongoDB

### 7.8.4 Automatisierte Generierung mehrerer Datensätze

Um größere Datenmengen zu erzeugen, kann die KI mehrfach aufgerufen oder mit variierenden Parametern gefüttert werden. Anschließend werden die generierten Datensätze gesammelt und in die Datenbank eingefügt.

Ein vereinfachtes Beispiel:

```
const dummyArray = [ /* mehrere KI-generierte Objekte */ ];

await collection.insertMany(dummyArray);
```

**Listing 3:** Batch-Insert mehrerer Dummy-Daten

### 7.8.5 Validierung der generierten Daten

Da KI-generierte Daten potenziell inkonsistent sein können, ist eine Validierung erforderlich. Diese kann durch:

- Schema-Validierung in MongoDB
- Backend-Validierung (z. B. mit JSON-Schema)
- Plausibilitätsprüfungen (z. B. kompatible GPU für macOS)

realisiert werden.

### 7.8.6 Vorteile des KI-Einsatzes

Der Einsatz von KI zur Dummy-Datengenerierung bietet mehrere Vorteile:

- Schnelle Erstellung großer Datenmengen
- Realitätsnahe Simulation von Hackintosh-Systemen
- Reduzierung manueller Eingaben
- Hohe Variabilität der Datensätze

### 7.8.7 Grenzen und Herausforderungen

Trotz der Vorteile bestehen auch Einschränkungen:

- Mögliche Inkonsistenzen in den generierten Daten
- Notwendigkeit einer Nachbearbeitung
- Abhängigkeit von der Qualität des Prompts

### 7.8.8 Fazit

Die Nutzung von KI zur Generierung von Dummy-Daten stellt eine effiziente Methode dar, um die Hackintosh-Datenbank zu testen und zu evaluieren. In Kombination mit MongoDB ermöglicht dies eine flexible und skalierbare Datenbasis, die realistische Anwendungsszenarien abbildet.

# 8 Evaluation

## 8.1 Testdaten

Zur Evaluation wurden 50 Dummy-Geräte generiert. Die Datensätze umfassen unterschiedliche Hersteller, CPUs, GPUs und macOS-Versionen.

## 8.2 Performance

Abfragen in MongoDB werden innerhalb von Millisekunden ausgeführt, auch bei großen Datenmengen. Die Kombination mit Indexen auf ‘device$_name$‘$ermöglichtschnelleSuchen$.

## 8.3 Ergebnisse

- Webfrontend erlaubt schnelle Eingabe neuer Geräte
- Suchen nach Geräten liefert vollständige Hardwareinformationen
- Datenbankstruktur ist flexibel und erweiterbar

# 9 Fazit

Diese Arbeit hat gezeigt, dass eine MongoDB-basierte Datenbank die Verwaltung von Hackintosh-Konfigurationen erheblich erleichtert. Die Webanwendung ermöglicht effiziente Dateneingabe, Abfrage und Analyse von Geräten.

## 9.1 Zusammenfassung

- Strukturierte Speicherung von Hardwareinformationen
- Wiederverwendbarkeit von OpenCore-Konfigurationen
- Schnelle Abfrage und Filterung von Geräten

## 9.2 Ausblick

Zukünftig könnten folgende Erweiterungen implementiert werden:

- Benutzerverwaltung für mehrere Administratoren
- Automatisierte Aktualisierung von Kext-Versionen
- Visualisierung von Kompatibilitätsstatistiken

### Device Queries

```
// Device Collection
db.Device.insertOne({
    device_id: ObjectId(),
    device_name: "Hackintosh Pro X",
    manufacturer: "Generic Inc.",
    cpu: "Intel i9-12900K",
    gpu: "NVIDIA RTX 3080",
    wifi: "Broadcom BCM94360",
    status: "active",
    opencore_version: "0.9.3",
    config_plist: "/configs/hackintosh_prox.plist"
```

```
});

db.Device.find({ status: "active" });

db.Device.updateOne(
    { device_name: "Hackintosh Pro X" },
    { $set: { status: "inactive" } }
);
```

**Listing 4:** Device Collection Queries

### MacOS_Version Queries

```
// MacOS_Version Collection
db.MacOS_Version.insertMany([
    { version_id: ObjectId(), version_name: "macOS Ventura 13.4" },
    { version_id: ObjectId(), version_name: "macOS Monterey 12.6" }
]);

db.MacOS_Version.find({ version_name: /Ventura/ });

db.MacOS_Version.updateOne(
    { version_name: "macOS Monterey 12.6" },
    { $set: { version_name: "macOS Monterey 12.6.1" } }
);
```

**Listing 5:** MacOS Version Collection Queries

### Kext Queries

```
// Kext Collection
db.Kext.insertMany([
    { kext_id: ObjectId(), name: "Lilu.kext" },
    { kext_id: ObjectId(), name: "WhateverGreen.kext" }
]);

db.Kext.find({ name: "Lilu.kext" });

db.Kext.updateOne(
    { name: "WhateverGreen.kext" },
    { $set: { name: "WhateverGreen-Updated.kext" } }
```

```
);
```

**Listing 6:** Kext Collection Queries

### ACPI_Patch Queries

```
// Kext Collection
db.Kext.insertMany([
    { kext_id: ObjectId(), name: "Lilu.kext" },
    { kext_id: ObjectId(), name: "WhateverGreen.kext" }
]);

db.Kext.find({ name: "Lilu.kext" });

db.Kext.updateOne(
    { name: "WhateverGreen.kext" },
    { $set: { name: "WhateverGreen-Updated.kext" } }
);
```

**Listing 7:** ACPI Patch Collection Queries

### BIOS_Settings Queries

```
// BIOS_Settings Collection
db.BIOS_Settings.insertOne({
    bios_id: ObjectId(),
    secure_boot: false,
    vt_d: true
});

db.BIOS_Settings.find({ secure_boot: false });

db.BIOS_Settings.updateOne(
    { bios_id: ObjectId() },
    { $set: { secure_boot: true } }
);
```

**Listing 8:** BIOS Settings Collection Queries

### Known_Issue Queries

```
// Known_Issue Collection
db.Known_Issue.insertMany([
    { issue_id: ObjectId(), description: "Sleep mode causes kernel panic" }
    { issue_id: ObjectId(), description: "WiFi not recognized on boot" }
]);

db.Known_Issue.find({ description: /WiFi/ });

db.Known_Issue.updateOne(
    { description: "Sleep mode causes kernel panic" },
    { $set: { description: "Sleep mode kernel panic fixed in patch" } }
);
```

**Listing 9:** Known Issue Collection Queries

[**dortania**]

# Abbildungsverzeichnis

1 Übersicht der Systemarchitektur . . . . . 15

2 Übersicht des Backends und Datenflusses zwischen Frontend und Datenbank . . . . . 21

3 Gesamtübersicht des implementierten Systems . . . . . 26

# Tabellenverzeichnis

1 Felder des Device-Modells für Hackintosh-Konfigurationen . . . . . 13

# Listings


1 Beispiel eines KI-generierten Dummy-Datensatzes . . . . . 28
2 Einfügen eines KI-generierten Datensatzes in MongoDB . . . . . 29
3 Batch-Insert mehrerer Dummy-Daten . . . . . 30

4 Device Collection Queries . . . . . 33
5 MacOS Version Collection Queries . . . . . 34
6 Kext Collection Queries . . . . . 34
7 ACPI Patch Collection Queries . . . . . 35
8 BIOS Settings Collection Queries . . . . . 35
9 Known Issue Collection Queries . . . . . 36

# Abkürzungsverzeichnis

DoS Denial of Service. *Glossar:* Denial of Service

## Selbstständigkeitserklärung

Hiermit erkläre ich, dass ich die vorliegende Arbeit selbständig verfasst und keine anderen als die angegebenen Hilfsmittel benutzt habe. Die Stellen der Arbeit, die anderen Quellen im Wortlaut oder dem Sinn nach entnommen wurden, sind durch Angaben der Herkunft kenntlich gemacht. Dies gilt auch für Zeichnungen, Skizzen, bildliche Darstellungen sowie für Quellen aus dem Internet.

Ich erkläre ferner, dass ich die vorliegende Arbeit in keinem anderen Prüfungsverfahren als Prüfungsarbeit eingereicht habe oder einreichen werde.

Die eingereichte schriftliche Arbeit entspricht der elektronischen Fassung. Ich stimme zu, dass eine elektronische Kopie gefertigt und gespeichert werden darf, um eine Überprüfung mittels Anti-Plagiatssoftware zu ermöglichen.

Wismar, 05.04.2026

Ort, Datum

Unterschrift

# Thesen

**Hackintosh-Datenbank Dokumentation**

**Datenbankgestützte Verwaltung von Hackintosh-Konfigurationen: Analyse und Implementierung mit MongoDB**


| | |
|---|---|
| Eingereicht am: | 05.04.2026 |
| von: | Kilian Ketelhohn (520858)<br>geboren am 27.11.2001 in Schwerin |
| Betreuer: | Prof. Dr. Antje Raab-Düsterhöft |


- Eine strukturierte MongoDB-Datenbank erleichtert die Verwaltung von Hackintosh-Konfigurationen
- Automatisierte Erfassung und Wiederverwendung von OpenCore-Settings reduziert den Installationsaufwand
- Flexibles Datenmodell erlaubt einfache Anpassung an unterschiedliche Hardware
- Zentrale Wissensbasis steigert die Erfolgsquote von macOS-Installationen
- Analyse von Hardwarekompatibilität ermöglicht systematische Optimierungen
- Einsatz einer Webanwendung (React + Node.js) erleichtert Eingabe und Suche von Geräten
- Dummy-Daten und Testabfragen zeigen die Praktikabilität des Datenbankmodells